

AI-Assisted Legacy App Migration Playbook

Oct 3, 2026 · ERM Slate to React migration

Summary

Two developers rebuilt the ERM Risk & Opportunity Dashboard from Foundry Slate to React and OSDK in one SAFe PI, with no documentation to start from. The app is ready for go-live, and an AI-assisted method made it possible; this playbook makes that method repeatable for any other legacy app.

The method has seven steps, from packing the whole application into AI-ready context with our `pack-project.ts` script to producing architecture and lifecycle diagrams. On ERM it recovered 105 legacy features, found 31 gaps (7 of them security or data-integrity issues), and added 11 features Slate never had. Against a generic manual estimate it saved about 76% of the effort, roughly \$80K.

The challenge

The legacy app was large, spread across many files, and documented nowhere. Its behaviour lived only in Slate configuration and in the heads of its users.

Aspect	Legacy Slate app
Structure	1 shell app + 7 modules: print preview, action tracker, full search, heatmap, one-pager, report table, waterfall
Source size	8 Slate JSON exports, about 25 MB of widgets, variables, events, queries and embedded iframe code
Documentation	None: no specs, no diagrams, no feature list
Users	892 lifetime, 320+ in a peak month, 1,600+ dashboards, across 6 countries and many organisations
Data and security	ARM risk data behind restricted views; Foundry functions and actions run as the user
Team and timeline	2 developers, one SAFe PI, a target the team set itself

On top of parity, we wanted the React app to add new features rather than just copy the old one.

Step 0: Build AI context with pack-project.ts

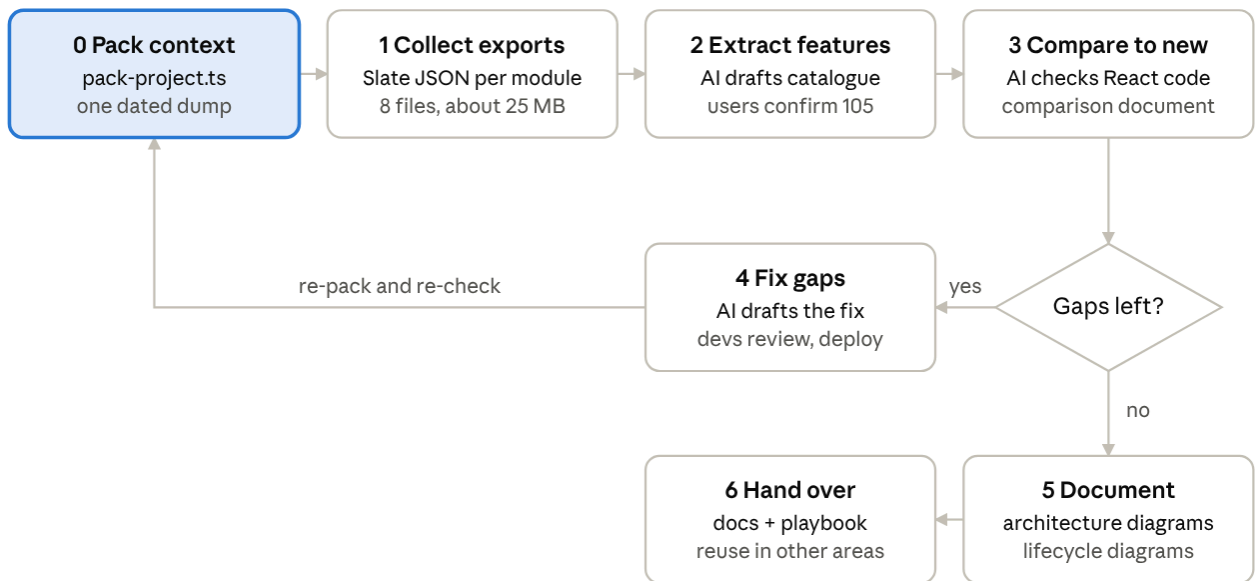
An AI model can only reason about what it is given. `pack-project.ts` is a small script we wrote that turns a whole codebase into one file the model can read in one go.

- **What it does:** walks the project folder and writes every source file into a single `PROJECT FILE DUMP`. Each file sits under a `FILE PATH:` header, and the dump is stamped with the time it was generated.
- **ERM example:** the React app packed into one text file of 75 source files, about 0.8 MB.
- **Why it matters:** the model sees the components, hooks, OSDK calls and types together, so its answers trace real code paths instead of guessing from fragments. Re-running the script after every change gives a fresh, dated snapshot, so the model never works from stale code.
- **Legacy side:** the Slate apps needed no packing, because each Slate export is already one self-contained JSON file per module.

The packed React dump plus the eight Slate exports are the complete context for every later step.

The playbook

Seven steps take a legacy app from undocumented to migrated and documented; steps 3 and 4 repeat until no unplanned gaps remain.

Fix and re-check until no gaps are left, then document

migration playbook · 7 steps, 1 loop

Each fix is re-packed and re-checked, so the comparison document always reflects the current code.

Step	Input	Output	Who checks
0 Pack context	New codebase	One dated dump from <code>pack-project.ts</code>	Developers
1 Collect exports	Legacy app	One JSON export per Slate module	Developers
2 Extract features	Exports	Feature catalogue (105 for ERM)	Business users
3 Compare to new	Catalogue + dump	Comparison document: parity, new features, gaps	Business users + developers
4 Fix gaps	Each gap	Reviewed, tested, deployed fix	Developers + function owners
5 Document	Final dump + exports	Architecture and lifecycle diagrams	Whole team
6 Hand over	All deliverables	Documented app; playbook for the next team	Team lead

Deliverables produced for ERM

Every step left a reusable artifact behind; together they form the documentation the app never had.

Deliverable	What it gives the team	Step
Packed React context	One dated file of the whole React codebase (75 files) for AI review	0
Feature catalogue	All 105 legacy features, grouped by module	2
Feature comparison document (PDF)	Each legacy feature against React, the 11 new React features, and any gaps	3
Gap register	31 gaps found and prioritised, 7 of them security or data-integrity issues	3
Backend fix to <code>executeAddEditWorkflow</code>	Board members get access to risks when a dashboard is created or edited (deployed)	4
React fixes	App-wide toast notifications and dashboard list refresh	4
11 new features	Permissions-only edit, a link per tab, dashboard switcher, new filters, one-pager layout 4, resizable columns, report classification, slideshow and more	4
Architecture diagrams	Detailed views of the Slate app and the React app	5
Lifecycle diagrams	Dashboard lifecycle (Slate, React) and risk lifecycle (combined, React only)	5

Impact in numbers

The hard facts carry the case; the cost figures are generic estimates and are labelled as such.

Measure	Value	Basis
Time and team	1 SAFe PI, 2 developers	Actual
Features carried over	105, each checked against the code	Actual

Measure	Value	Basis
Gaps found by AI review	31, of which 7 security or data-integrity	Actual
New features Slate never had	11	Actual
Code size	25 MB of Slate configuration → 0.8 MB of typed code (~97%)	Actual
Documentation	0 → 10 documents (8 diagrams, 2 guides)	Actual
Effort saved	~76%: 4 sprints actual vs ~17 sprints estimated	Estimate: 105 features × 3 days + 25 days docs ≈ 340 person-days; 2 developers = 20 person-days per sprint
Cost saved	~\$80K: ≈ \$25K actual vs ≈ \$105K manual	Estimate at \$38.50 per developer hour, 8 hours a day
Recurring cost	~\$10K a year less maintenance	Estimate: maintenance ≈ 20% of build effort a year (68 person-days ≈ \$21K), assumed halved by modular, typed, documented code

Teamwork and cross-functional cooperation

With no documentation, no single group could rebuild the app alone; the playbook gave each group a clear part to play.

- **Business and developers rebuilt the knowledge together.** The AI drafted the 105-feature catalogue; [ERM business users / risk officers] confirmed it against how they really use the tool. The comparison document became the shared answer to "does the new app do X?".
- **Data and security owners were involved early.** ARM risk data sits behind restricted views. With [ARM data owners / platform team] we kept that model and fixed how access is granted when a dashboard is created or edited.
- **Backend and frontend moved together.** Function and UI fixes were reviewed and deployed together with [backend function owners].
- **Diagrams gave everyone one picture.** Business, development and support now describe the system the same way.

What the business says (ERM stakeholders, September 2026; about the team and the tool overall):

- “Thanks to a mix of regular teamwork with the business and strong IT skills, they have turned an idea into a user-friendly industrial [solution]... User interface and performance have significantly improved.” (ERM Centre of Competence)
- “The Skywise ERM dashboard application is a tremendous addition to our ERM toolset... More and more users regularly prepare and facilitate risks & opportunities reviews with the application.” (ERM stakeholder, [Role])
- “They built great new features that make navigation easier and give us a much clearer view of our R&O data. Whenever bugs or issues came up, they fixed them quickly and efficiently.” (Risk and Opportunity Manager, Hamburg)

Open: name the teams and divisions involved and how many people took part.

Sustainability and transfer to other business areas

The playbook needs only the legacy app's export, the new codebase and an AI model, so it carries over to any team with a legacy app to replace.

Sustainable

- The app is now standard React and OSDK code: version-controlled, typed against the data model, and 97% smaller than the Slate configuration it replaces (0.8 MB against 25 MB).
- New features are fast to build: one isolated, typed component plus one OSDK call. 11 new features were built in the same PI.
- An undocumented app now has a feature catalogue, a comparison document and architecture and lifecycle diagrams, so new joiners and support staff can learn it quickly.
- The security model (restricted views, functions that run as the user) was kept, so no governance rework was needed.
- `pack-project.ts` keeps the AI context current: re-pack after each change and re-run the review. Estimated maintenance saving: about \$10K a year.

Transferable

- We have aligned with [Architect] to propose the playbook at the architecture sync, so functions beyond DMS can use it.
- Other Slate apps in [business areas] can follow the same seven steps unchanged.
- Other low-code or legacy tools with little documentation fit too: only the export in step 1 changes.

- `pack-project.ts`, the comparison document layout and the diagram formats are reusable as they are.

Jury demo script (about 10 minutes)

Show the method working, not just the result: each segment has one thing on screen.

Time	Slide (v2 deck)	Say
0:00–0:30	1 Cover	“2 developers rebuilt a business-critical, undocumented app for DevX in one PI, with an AI method any team can reuse.”
0:30–1:45	2 The stakes	“892 users, 1,600+ dashboards, many organisations in 6 countries: we could not break it.”
1:45–2:45	3 The complication	“Tangled Slate, no documentation. How do 2 developers move 105 hidden features in one PI?”
2:45–4:15	4 The insight	“AI reverse-engineers, people verify.” Show <code>pack-project.ts</code> ; optional 60-second live demo
4:15–6:00	5 The proof	“105 features, 31 gaps, 7 security issues caught, 11 new features. ~\$80K saved and ~\$10K a year less maintenance.”
6:00–7:00	6 Voice of the business	Read one quote aloud
7:00–8:30	7 Strategic fit	Walk the four 2026 objectives and the team strip
8:30–10:00	8 Scale and ask	“Endorse the playbook; we are proposing it at the architecture sync for functions beyond DMS.”

Before the demo:

- ☐ Fill in [Architect], [Role], [solution] and the team names
- ☐ Run the deck once in Slide Show to check animations and GIFs
- ☐ Prepare the live app for the optional demo on slide 4
- ☐ Be ready to explain that the effort, cost and maintenance figures are generic estimates

Lessons learned and guardrails

The AI speeds up every step, but people still own the decisions and the checks.

- **Complete context beats clever prompts.** Answers improved most once the model had the full packed codebase and every Slate module, not excerpts.

- **Humans confirm, AI drafts.** Business users signed off the feature catalogue; developers reviewed and tested every fix before deploying.
- **Declare deliberate differences.** Features left out by choice (the heatmap slider, report KPIs) are marked as such, so they are not reported as gaps.
- **Respect the data model's security.** Restricted views and run-as-user functions stay in place; fixes are checked against who can see which risk.
- **Use approved tools and channels.** Share code and data with AI only through tools and routes the company has approved.
- **Re-pack before each review.** A stale dump produces stale advice.